ENGINEERING REFERENCE

vdrapkinTimerManager

Windows RTOS-Style Timer Manager

Architecture, Protocol, Implementation, Sample Application, and Verification

---

A console-based Windows reference implementation of a client/server timer service, including its public interface, communication protocol, scheduling algorithm, sample application, and automated tests.

Prepared by Vitaly Drapkin

Version 1.4 | July 5, 2026

Document Purpose

This manual describes the complete vdrapkinTimerManager project. It introduces the externally visible timer behavior first, then defines the participating components, communication protocol, scheduling algorithm, server processing, sample application, and synchronization rules. The final sections describe the build procedure and the automated tests that verify normal operation, rejected requests, capacity limits, concurrency, and arithmetic boundaries.

Contents

1. Overview

vdrapkinTimerManager provides an RTOS-style timer service for Windows applications. In this manual, an application is a client program that uses the service, and a logical timer is a service-managed object identified by a numeric timer ID. Each logical timer is one-shot: one successful start can produce at most one expiry, and the service does not restart the timer automatically.

The public C API defines the timer lifecycle. CreateTimer creates a logical timer and returns its timer ID without starting a countdown. StartTimer starts a countdown using a duration in milliseconds. If the application calls StartTimer again before that timer expires, the service cancels the current countdown and starts a new countdown from the time of the second call using the new duration. StopTimer cancels a running countdown without deleting the timer, and DeleteTimer releases the timer ID and its associated resources.

CreateTimer, StartTimer, StopTimer, and DeleteTimer are synchronous commands: each function waits until the service returns an acknowledgment (ACK) containing the command result. Expiry delivery is asynchronous. The API provides a notification event, which is a Windows wait handle that becomes signaled when at least one expired timer ID is available. After the event is signaled, the application calls GetExpiredTimer until the first-in, first-out (FIFO) expiry queue is empty.

The API implementation is a client library linked with the application. It communicates with the vdrapkinTimerManager console server through \\.\pipe\vdrapkinTimerManager, a Windows named pipe used as the interprocess communication channel. The server supports at most 61 simultaneous client connections, and the public API may be called by multiple application threads. Figure 1 shows these externally visible components. Later sections define the protocol, scheduling data structures, Windows wait objects, and synchronization used to implement this behavior.

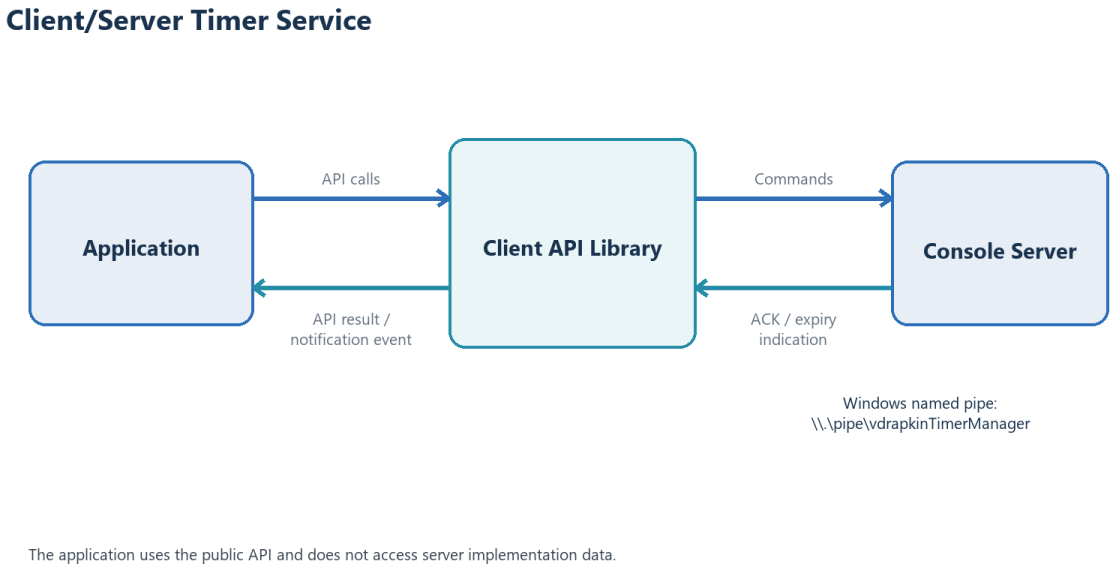

Figure 1. Client/server timer-service overview

2. Scope and Design Principles

The project is intentionally small enough to study from end to end, yet complete enough to exercise timer ownership, asynchronous delivery, cleanup, and failure handling. Its design choices favor visible control flow and deterministic behavior over production-scale throughput.

2.1 Included

The implemented lifecycle covers creation, start or restart, stop, expiry, and deletion. Each timer is associated with the client connection that created it, so another client cannot operate on that timer and disconnect cleanup can release all timers associated with the lost connection.

The project includes asynchronous client/server communication, event-based expiry notification, orderly Ctrl+C shutdown, and a sample connection state machine driven by timer expiries. Unit, integration, robustness, capacity, concurrency, and sample-specific tests verify the documented behavior.

2.2 Deliberate Exclusions

The example supports one-shot timers only. It does not add periodic scheduling, callback-based application delivery, persistent timer storage, or automatic server startup because those features would obscure the request, wait, and expiry paths that the sample is meant to expose.

The server also avoids I/O completion port (IOCP) worker threads, a separate scheduler thread, Windows service hosting, authentication, authorization, and cross-version negotiation. Timer allocation is limited by process memory rather than a configured timer count, and allocation failure is reported through the public API as an internal error. These boundaries keep the implementation educational rather than presenting it as a commercially hardened service.

2.3 Coding Contract

Each function validates all parameters before entering its operational logic. A programmer-contract violation asserts in a Debug build when execution cannot continue safely, while failures that a caller can handle are returned through the appropriate status value. The implementation keeps its RTOS-style C structures, uses malloc and free for dynamic ownership, avoids goto and calloc, separates logical blocks with whitespace, and identifies every function at its closing brace so control flow remains easy to follow.

3. Public Client API

The public API converts the lifecycle described in Section 1 into six callable functions. Four command functions create, start, stop, or delete a timer and return an ACK status. Two notification functions provide the expiry event and retrieve queued timer IDs. Table 1 specifies the exact function names, returned information, and caller obligations.

Table 1. Public client API

API	Behavior	Important rule
vdrTimerManager_CreateTimer	Creates one service-managed logical timer without starting its countdown and returns its nonzero ID.	This must be the first public call because the library performs its initialization automatically during this call.
vdrTimerManager_StartTimer	Starts a countdown for a timer that is not running or restarts a running timer with a new duration.	The duration_ms argument must be nonzero, and the caller waits for the server ACK.
vdrTimerManager_StopTimer	Cancels the running countdown without deleting the logical timer.	Stopping a timer that is not running succeeds and has no additional effect, making the operation idempotent.
vdrTimerManager_DeleteTimer	Deletes the logical timer.	A successful delete makes the timer ID invalid for later commands.
vdrTimerManager_GetNotificationEvent	Returns the API-owned	The application may wait on the

API	Behavior	Important rule
	notification event described in Section 1.	handle but must not close it.
vdrTimerManager_GetExpiredTimer	Removes one expired timer ID from the FIFO queue.	The API resets the event after the final queued ID is removed.

3.1 ACK Status Values

Each command returns one vdrAckStatus_type value to report success or the reason for failure. When a command concerns an existing timer, the response also identifies that timer. If timer creation fails before an ID is allocated, the API returns TIMER_ID_INVALID, whose reserved numeric value is zero. Table 2 defines every ACK status and its numeric value.

Table 2. ACK status values

Status	Value	Meaning
OK	0	The command completed successfully.
INVALID_TIMER_ID	1	The timer ID is zero or does not exist.
INVALID_DURATION	2	The requested start duration is zero.
TIMER_NOT_OWNED	3	The timer exists but belongs to another client pipe.
SERVER_BUSY	4	The value is reserved for future capacity or back-pressure behavior.
PROTOCOL_ERROR	5	The message header, type, byte count, or payload metadata is invalid.
INTERNAL_ERROR	6	Allocation, a Windows API, or an unexpected internal operation failed.
SERVER_NOT_AVAILABLE	7	The client cannot establish or continue pipe communication.
NO_EXPIRED_TIMERS	8	The client expiry queue is currently empty.

4. System Architecture

Timer processing is divided among four components. The application calls the public functions defined in Section 3. The client API converts each command call into a request and converts each response into an API result. The console server validates requests and coordinates communication with scheduling. The timer core is the server module that calculates when each

logical timer expires. Table 3 assigns each responsibility to one component so that ownership is not duplicated.

Table 3. System components and responsibilities

Component	Primary responsibility	Concurrency
Application	Calls the public API, assigns meaning to each timer, and combines the expiry event with its own wait handles.	The application chooses its threading model. The sample keeps state-machine logic on one main thread.
Client API	Converts public function calls into protocol requests and converts server responses into command results or queued expiry IDs.	The library serializes command exchanges and manages asynchronous expiry delivery. Section 9.1 defines the internal thread and lock behavior.
Console server	Owns pipes, validates protocol messages, dispatches timer operations, and sends ACK or expiry messages.	One main thread performs request and timer work. The console handler only signals shutdown.
Timer core	Stores internal scheduling state and calculates the next logical timer expiry.	The core has no internal lock because the server serializes every call.

4.1 Ownership Boundaries

The server owns the internal timer object created for each successful CreateTimer command and associates that object with the client connection that requested it. The timer core may update the object's scheduling state, but only the server communicates the command result or expiry indication to the client.

The client API library owns the connection and the notification event returned to the application. The application may include that event in a wait operation, but it must not close the handle. The library also owns all internal resources used to match command responses and retain timer IDs until the application retrieves them.

5. Shared Binary Protocol

The named-pipe protocol uses fixed-size C structures with compiler-native alignment. The client and server are built from the same protocol header and compiler application binary interface (ABI), so both executables use the same field offsets and structure sizes. A layout change

therefore requires both executables to be rebuilt together. Every request, ACK, and expiry indication begins with vdrMessageHeader_type, the common header structure listed in Table 4. The request union and response union each place that header first so the receiver can validate it before accessing message-specific fields.

Table 4. Message header fields

Header field	Type	Meaning
magic	uint32_t	Contains 0x544D4752 ('TMGR') so the receiver can reject unrelated bytes.
version	uint16_t	Contains protocol version 1 so incompatible endpoints fail before payload access.
message_type	uint16_t	Contains a vdrMessageType_type numeric value that selects the valid message structure.
request_id	uint32_t	Correlates one command with its ACK. Asynchronous expiry events use zero.
payload_size	uint32_t	Reports the bytes following the common 16-byte header for exact-size validation.

```
typedef struct vdrMessageHeader_tag {
    uint32_t magic;
    uint16_t version;
    uint16_t message_type;
    uint32_t request_id;
    uint32_t payload_size;
} vdrMessageHeader_type;
```

5.1 Message Catalog

After validating the common header, the receiver uses message_type to select one of six values from vdrMessageType_type, the enumeration that identifies every protocol message. The numeric values are part of the shared protocol contract, and the received byte count must match the fixed structure size associated with the selected type. Table 5 lists the mapping among symbolic message names, numeric values, structure sizes, and payload fields.

Table 5. Protocol message catalog

Direction	Message	Value	Bytes	Payload
Client to server	CREATE_TIMER_REQ	1	16	Contains only the common header.
Client to server	START_TIMER_REQ	2	32	Contains timer_id and duration_ms.
Client to server	STOP_TIMER_REQ	3	24	Contains timer_id.

Direction	Message	Value	Bytes	Payload
Client to server	DELETE_TIMER_REQ	4	24	Contains timer_id.
Server to client	ACK_RSP	5	32	Contains status, reserved, and timer_id.
Server to client	TIMER_EXPIRED_EVT	6	24	Contains the expired timer_id.

5.2 Validation Contract

The server validates each message in a fixed sequence. It first requires a complete header, then checks the magic value and protocol version, and then verifies that message_type identifies a supported client command. After identifying the command type, the server requires the exact structure size and verifies that payload_size equals the number of bytes following the common header.

Command logic reads timer IDs or durations only after all header and size checks succeed. For a malformed request, the server returns an ACK whose status is PROTOCOL_ERROR, the protocol-validation status defined in Table 2, and whose timer ID is the reserved TIMER_ID_INVALID value. A protocol validation failure rejects only the current request and does not close the client connection, so the client may send a corrected request afterward. This validation order prevents untrusted bytes from selecting the wrong union member and verifies that rejecting one message does not make the connection unusable.

MALFORMED MESSAGE BEHAVIOR: For a malformed request, the server returns PROTOCOL_ERROR with TIMER_ID_INVALID before executing payload logic. The server keeps the same client connection open because the error applies to the request rather than to the connection. The robustness test then sends a valid command through that connection to verify continued operation.

The client receiver independently validates every server response before accessing the server-to-client message union. After requiring a complete header and checking the magic value and protocol version, it accepts only ACK_RSP or TIMER_EXPIRED_EVT. The selected type determines the exact expected structure size; the received byte count must equal that size, and payload_size must equal the bytes following the common header. Any unsupported type, truncated or oversized message, or payload-size mismatch is ignored before another read is posted. An invalid server message therefore cannot satisfy a command waiting for an ACK and cannot add a timer ID to the application expiry queue.

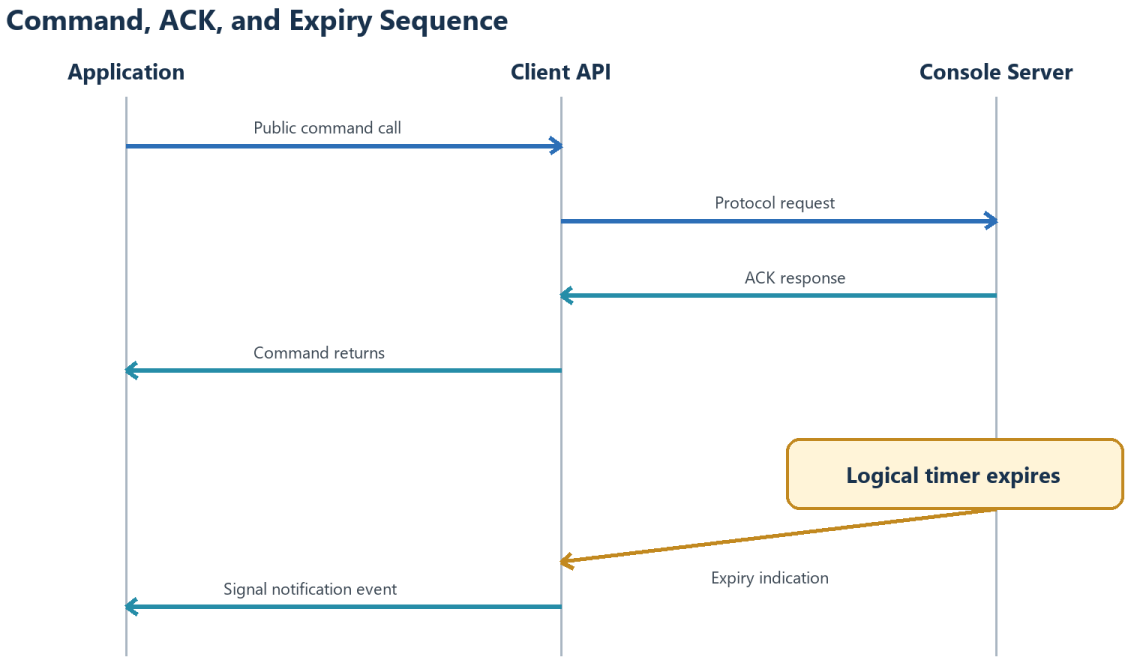

Figure 2. Synchronous command/ACK and asynchronous expiry delivery

6. Timer Core

The timer core implements the scheduling behavior after the API and protocol have accepted a command. For each logical timer, the core allocates one Timer_type structure containing the timer ID, owner connection, duration, timing values, and the next_timer link pointer used to place the structure in a singly linked timer list. A created, stopped, or expired timer is stored on the inactive list, which contains timers without a running countdown. A running timer is stored on the active list, which the core orders by expiry time. Instead of allocating one Windows timer per logical timer, the server programs one Windows waitable timer, a kernel wait object that becomes signaled at a specified time, for the next logical expiry. The server supplies the current time and an optional expiry callback, a server function that the core calls after a timer expires.

Within Timer_type, TimerId_type is the project's unsigned 64-bit timer-ID type, and server_pipe is the Windows HANDLE for the owning client connection. The duration_ms field stores the requested duration, and the system_start_time_ms field stores the timestamp supplied when the timer was started. The timer's scheduled expiry is system_start_time_ms plus duration_ms. For the first timer in the active list, time_remainder_ms stores the remaining delay calculated as max(scheduled expiry - now_ms, 0), where now_ms is the current timestamp supplied when the core recalculates the list. It is not another copy of the timer's start timestamp. For every

later active timer, time_remainder_ms stores the difference between that timer's scheduled expiry and the preceding timer's scheduled expiry. The next_timer link pointer identifies the next structure in the active or inactive singly linked list.

```
typedef struct Timer_tag {
    TimerId_type id;
    HANDLE server_pipe;
    uint64_t duration_ms;
    uint64_t system_start_time_ms;
    uint64_t time_remainder_ms;
    struct Timer_tag* next_timer;
} Timer_type;
```

6.1 Active and Inactive Lists

A timer begins in the inactive list immediately after creation. Start removes it from the inactive list and inserts it into the active list according to its scheduled expiry, which is the start time plus the requested duration. Starting an already active timer removes the existing active-list entry and reinserts the same allocation using the new start time and duration.

Stop and expiry return the allocation to inactive state, which allows the application to start the timer again. Delete removes the timer from whichever list owns it and frees the allocation. When a pipe disconnects, the core applies that same deletion rule to every timer whose owner matches the lost pipe, leaving timers owned by other clients untouched. Removing an active timer can invalidate the stored delta of a timer that followed it, so disconnect cleanup records whether the active list changed and recalculates all remaining active-list deltas after every owned timer has been freed. The cleanup function has no current timestamp; its recalculation uses zero to reconstruct the differences between surviving scheduled expiries. Before the server arms the waitable timer, GetNextWaitDuration updates the active head against the actual current timestamp. This sequence preserves every surviving timer's original scheduled expiry when cleanup removes a timer from the head, middle, or tail of the active list.

6.2 Delta Representation

The active list is ordered by absolute scheduled expiry, but each node stores a relative delay. Whenever the core recalculates the list, it receives now_ms as the current server timestamp. For the first active timer, the core sets time_remainder_ms to max(scheduled expiry - now_ms, 0), which is the remaining time until that timer expires at the moment of recalculation. For every later timer, the core sets time_remainder_ms to the difference between that timer's scheduled expiry and the preceding timer's scheduled expiry. Timers with the same scheduled

expiry therefore store a zero delta, so one waitable-timer signal causes the server to process every timer due at that instant.

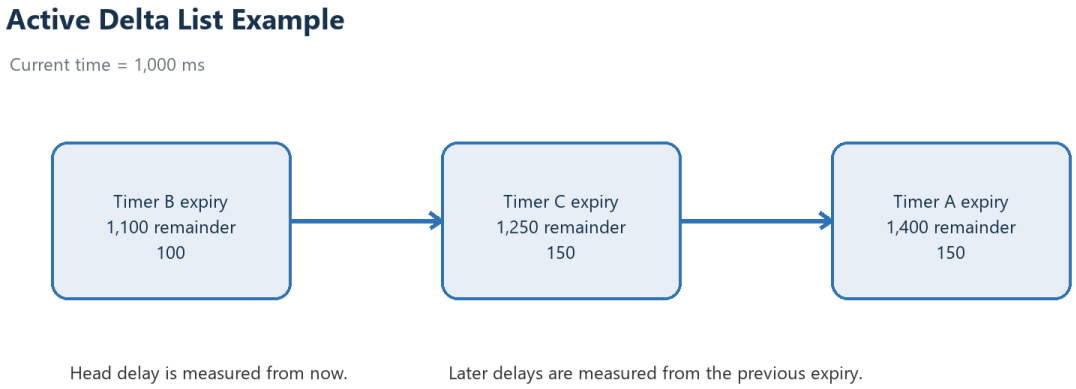

Figure 3. Scheduled expiry ordering and delta values

6.3 Expiry and Overflow

When the waitable timer signals, vdrTimerCore_ProcessExpired repeatedly removes the first active-list entry whose scheduled expiry has arrived and moves its allocation to the inactive list. The optional server callback runs only after the lists are consistent, so an expiry indication is not sent while the timer still appears active. Scheduled expiry uses saturating uint64_t addition, which clamps start time plus duration to UINT64_MAX instead of wrapping into the past. Boundary tests verify both saturation and ordering near the maximum timestamp.

7. Server Implementation

The server performs request dispatch and timer processing on one main thread. That thread owns the connected-client collection, timer core, waitable timer, and shutdown event. The shutdown event is a manual-reset Windows event that the console handler signals when Ctrl+C occurs. Sequential dispatch prevents simultaneous updates to timer-core state. Section 9 defines how these process-lifetime objects are grouped and accessed.

Named-pipe connection and read operations use Windows overlapped I/O, which allows an operation to remain pending while the main thread waits for other activity. Each pending operation has a completion event. The main thread passes those events, the waitable timer, and the shutdown event to WaitForMultipleObjects, a Windows function that blocks until one handle in an array becomes signaled. Because the main thread dispatches all three event classes sequentially, the design does not require a separate scheduler thread. Writes may wait

for completion because traceable control flow has priority over throughput in this RTOS simulation. A production server could instead use fully overlapped writes.

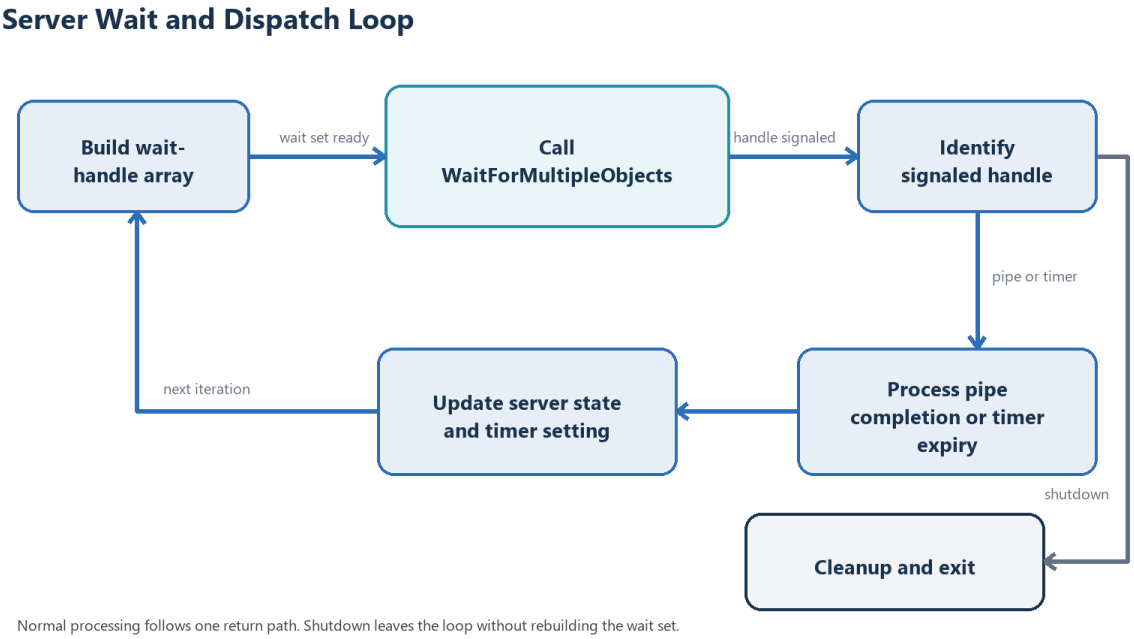

Figure 4. Main server wait-set construction and dispatch

7.1 Startup

Startup creates dependencies in the reverse order from shutdown cleanup. The server first initializes the timer core, then creates the waitable timer and manual-reset shutdown event. After those handles exist, it installs the console control handler and creates the initial listener, which is the named-pipe instance reserved for accepting the next client connection. The server enters its main loop and prints the readiness trace only after the listener's overlapped connection request is represented in the wait set.

7.2 Wait-Set Construction

At the beginning of each loop iteration, the server copies every live pipe event into a local handle array. It then asks the timer core whether an active head exists. When one does, the waitable timer handle for that head is appended to the array. The shutdown event is always appended last. Capturing the timer and shutdown indexes during construction lets the returned wait index identify a pipe, an expiry, or shutdown without maintaining a second dispatch table.

7.3 Listener and Capacity Handling

When a listener accepts a connection, that same pipe instance becomes a client and is armed for one asynchronous read. The server then creates a replacement listener so another connection can arrive while existing clients remain active. At 61 connected clients, the reserved listener occupies the final pipe slot. An excess connection is disconnected and the listener is rearmed instead of being promoted. If a logical timer is active at that moment, 62 pipe events plus the waitable timer and shutdown event consume exactly 64 handles. A listener can nevertheless be lost when listener rearming fails, the first read for an accepted client cannot be armed, a client read completes with an error, or the next client read cannot be armed. After removing the affected pipe, the server calls vdrEnsureListenerAvailable. That helper scans the compact pipe array and creates and arms a listener only when none is present, which restores the accept path after a capacity slot becomes available without creating duplicate listeners. A restoration failure is traced while existing clients continue to run; a later pipe-removal path makes another restoration attempt.

7.4 Client Commands

A client completion event identifies exactly one finished read. The server validates the message header and size before dispatching the selected timer-core operation, then sends a generic ACK containing the resulting status and timer ID. Because start, stop, delete, or disconnect may change the first timer in the active list, the server recalculates the waitable timer before posting the client's next read. Each client has one Windows OVERLAPPED control structure and one inbound buffer, so allowing only one outstanding read prevents either object from representing two operations at the same time.

7.5 Expiry Callback

For each due timer, the core moves the existing Timer_type allocation from active to inactive before invoking the server callback with the timer ID and owner pipe. The callback locates the corresponding client and sends TIMER_EXPIRED_EVT immediately. This handoff avoids a temporary expiry list and ensures that an application may restart the timer as soon as it receives the indication.

7.6 Disconnect and Shutdown

A failed read or send is treated as a disconnect. Removing the pipe first deletes every active and inactive timer owned by that client, compacts the pipe array, and rearms the waitable timer if the active head changed. When Ctrl+C occurs, the console handler signals the shutdown event

instead of performing cleanup. WaitForMultipleObjects then returns the shutdown-event index, and the main thread closes every pipe, deinitializes the timer core, cancels and closes the waitable timer, closes the shutdown event, and exits.

8. Sample Connection State Machine

The sample uses the API-owned notification handle to drive a simulated connection protocol. Its vdrSampleConnectionState_type enumeration defines IDLE = 0, WAITING_FOR_CONNECTION_ESTABLISHMENT = 1, and CONNECTED = 2. CONNECTION_* messages have no payload. Colored trace output identifies each simulated transmission, reception, timer expiry, and state transition while the state machine repeats its operating cycle.

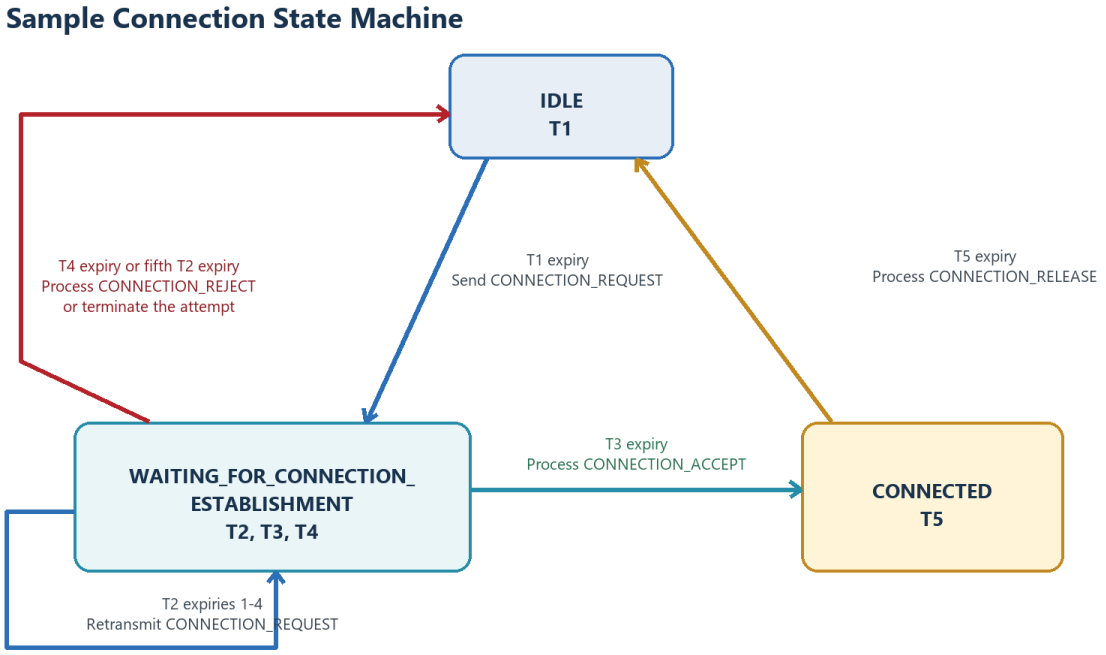

Figure 5. Timer-driven sample application state machine

Table 6. Sample application timers

Timer	Random duration	Meaning when the timer expires
T1	3 to 10 seconds	The application transmits CONNECTION_REQUEST, changes the state from IDLE to WAITING_FOR_CONNECTION_ESTABLISHMENT, and starts the connection-establishment timers.
T2	3 to 5 seconds	The application retransmits CONNECTION_REQUEST. After the fifth T2 expiry, the application terminates the attempt, changes the state to IDLE, and restarts T1.
T3	1 to 24 seconds	The application processes CONNECTION_ACCEPT, stops T4 and T2, changes the

Timer	Random duration	Meaning when the timer expires
		state to CONNECTED, and starts T5.
T4	1 to 24 seconds	The application processes CONNECTION_REJECT, stops T3 and T2, changes the state to IDLE, and restarts T1.
T5	5 to 15 seconds	The application processes CONNECTION_RELEASE, changes the state to IDLE, and begins the next cycle by starting T1.

In IDLE, the application starts T1 and remains in that state until T1 expires. It then transmits CONNECTION_REQUEST, changes the state to WAITING_FOR_CONNECTION_ESTABLISHMENT, and starts the connection-establishment timers. T2 controls retransmission, while T3 and T4 simulate acceptance and rejection after independently selected random delays. In CONNECTED, the application starts T5 and prints 'Data Exchange is ongoing...' once per second until T5 expires. The main thread waits on both the Timer Manager notification event and a Ctrl+C event, using bright yellow for messages, bright cyan for states, and bright magenta for timer expiries.

SIMULTANEOUS RESPONSES: T3 and T4 may both be queued when the application drains the shared expiry event. The application processes the first response, performs the corresponding state transition, and stops the peer timer. If the peer expiry is already queued, the application receives it after the state has changed and records it as stale. A deterministic self-test verifies exactly one state transition and one stale response.

9. Concurrency and Synchronization

Concurrency is controlled by assigning each shared resource both an owner and a synchronization rule. The server main thread serializes timer-core access, the client API uses internal locks for communication between application and receiver threads, and console handlers do not perform cleanup concurrently. Sections 9.1 and 9.2 define those rules before Table 7 summarizes them.

9.1 Client API Thread Safety

The client library creates one receiver thread to read ACK and expiry messages independently of application threads. It also stores a Windows INIT_ONCE structure so process-wide initialization runs exactly once even when several application threads call CreateTimer concurrently. That initialization creates three Windows critical sections, which are mutual-

exclusion locks used inside the library. Section 9.2 explains how functions obtain the shared client-state pointer before acquiring the required critical section.

The command_mutex lock permits one command and ACK exchange at a time. During that exchange, waiting_request_id stores the request ID expected by the command thread, and last_ack stores the matching ACK copied by the receiver thread. The ack_event object is the Windows event that the receiver thread signals after copying that ACK so the waiting command thread can continue. The state_mutex lock protects the connection, receiver lifecycle, request-matching fields, and ACK transfer. The expiry_mutex lock protects queue insertion and removal together with the notification-event state.

WHY LAST_ACK EXISTS The receiver thread reads all inbound pipe messages, while the thread executing a public command waits on ack_event. When the receiver obtains the ACK whose request ID matches waiting_request_id, it copies that ACK into last_ack while holding state_mutex and then signals ack_event. The command thread acquires state_mutex, reads last_ack, and returns the completed response to the caller.

9.2 Global State Access Rule

Each module groups its mutable process-lifetime data into one structure: vdrServerState_type for the server, vdrClientApiState_type for the client library, and corresponding state structures for the sample and tests. An accessor is a function that returns a pointer to its module's state object. Only vdrServer_GetState(), vdrClientApi_GetState(), SampleGetGlobalState(), or TestGetGlobalState() names the underlying global object directly. Every operational function calls the appropriate accessor once, stores the returned pointer locally, and accesses state members through that pointer.

An accessor defines the ownership boundary but does not lock the returned object. The server's single-thread rule and the client's critical sections still govern every read and update. Immutable static const lookup tables remain outside runtime state because they neither change nor participate in control-flow handoffs.

Table 7. Concurrency ownership and synchronization

Resource	Protection or owner	Why the rule is required
Public command path	command_mutex	Only one request may wait for one matching ACK at a time.
Lazy initialization	command_mutex and INIT_ONCE	Concurrent first calls must create one pipe, one receiver, and one event set.
ACK and pipe state	state_mutex	The receiver and command threads need a protected response and lifecycle handoff.

Resource	Protection or owner	Why the rule is required
Expiry FIFO	expiry_mutex	Receiver enqueue and application dequeue must preserve links and event state together.
Timer lists	Server main thread	The timer core contains no internal lock and relies on serialized callers.
Shutdown	Manual-reset event	The console handler wakes the main loop instead of performing cleanup concurrently.

The API receiver thread is created and owned by the client library rather than by the named-pipe mechanism. Windows completes overlapped operations and signals events, but application code controls the receiver's lifetime. That distinction matters during cleanup because the API must stop the loop, cancel its read, join the thread, and only then release the handles and queued expiry nodes that the thread could otherwise touch.

10. Error Handling and Defensive Behavior

Validation and error handling occur at each defined interface boundary. Public API functions reject TIMER_ID_INVALID and a zero duration before building a command. The server verifies request metadata before reading payload fields, checks timer ownership during lookup, and returns TIMER_NOT_OWNED when an otherwise valid ID belongs to another pipe. In the opposite direction, the client receiver accepts only an exactly sized ACK or expiry indication with matching protocol metadata; malformed server messages are ignored before they can affect command completion or expiry delivery.

Failures inside the implementation are translated into stable API outcomes. Allocation and unexpected Windows failures become INTERNAL_ERROR, while connection or communication loss becomes SERVER_NOT_AVAILABLE on the client. On the server, a pipe failure initiates disconnect cleanup so no timer remains associated with a dead owner. The expiry queue also reports an invalid indication or allocation failure instead of silently corrupting its FIFO state.

Rejected work must leave the existing system state usable. Protocol tests send a valid command after malformed input to verify continued communication. Stop and delete tests wait beyond the original deadline to verify that no stale expiry is delivered. Saturating timestamp arithmetic applies the same defensive rule at the numeric boundary by preventing a large duration from wrapping into an immediate expiry.

11. Building and Running

11.1 Prerequisites

The C11 project targets Windows 10 or later and is built with Visual Studio 2022, the MSVC desktop toolchain, and an installed Windows SDK. CMake 3.20 or later generates the solution and registers each test scenario with CTest. CMake configures every project translation unit to compile as C11 without compiler-specific language extensions.

11.2 Configure and Build

Run the following commands from the repository root. CMake creates an x64 Visual Studio solution in build, and the second command compiles the Debug configuration.

```
cmake -S . -B build -G "Visual Studio 17 2022" -A x64
cmake --build build --config Debug
```

Under MSVC, every C target compiles with /W4 and /WX. A compiler warning therefore stops the build instead of being accepted as successful output. Non-MSVC configurations use -Wall, -Wextra, and -Werror for the same reason.

11.3 Run

Start build\Debug\vdrapkinTimerManager.exe in the first console and wait until it reports that the named-pipe listener is armed. Then start build\Debug\vdrTimerManagerSampleClient.exe in a second console with no command-line arguments. The sample will print timer expiries, simulated messages, and state transitions as connection cycles repeat. Press Ctrl+C in the sample for orderly application cleanup, then stop the server with Ctrl+C.

11.4 Run Tests

```
ctest --test-dir build -C Debug --output-on-failure
```

The timer-core tests run without a server, while integration, robustness, and sample self-tests require the named-pipe endpoint. The regression workflow starts a temporary hidden server, executes every registered CTest scenario in a separate process where appropriate, and then stops the server. This lifecycle makes cleanup failures visible and prevents one test's client state from leaking into the next.

12. Verification Strategy

The regression suite contains 56 named CTest scenarios arranged in layers. Timer-core tests establish deterministic list and arithmetic behavior before any pipe is involved. Integration tests then exercise the real API and server, robustness tests challenge protocol, ownership, disconnect, capacity, and concurrency boundaries, and the sample self-test resolves simultaneous response expiries. Running scenarios independently isolates the client API's process-lifetime state and associates each documented requirement with a named result.

Table 8. Automated test suite composition

Layer	Count	Purpose
Timer-core unit	30	These tests establish deterministic list, scheduling, ownership, restart, cleanup, callback, and uint64_t behavior without pipe dependencies.
Client/server integration	9	These tests exercise the real API, named pipe, ACK handoff, expiry event, FIFO queue, cancellation, restart, and negative ACK behavior.
Robustness	16	These tests challenge malformed protocol input, exact client and wait-handle limits, capacity recovery, disconnect cleanup, ownership errors, and concurrent first API calls.
Sample self-test	1	This test forces simultaneous T3 and T4 expiries and verifies one transition plus one stale response.

12.1 Why Negative Tests Matter

A negative test verifies both the returned error code and the state after rejection. It checks that timer lists, ownership, wait duration, connection usability, and event state remain unchanged. The malformed-message tests issue a valid command after rejection to verify connection recovery. The stop and delete prevention tests wait beyond the original deadline to verify that cancellation prevents notification delivery.

12.2 Complete Automated Test Catalog

Table 9 maps each scenario name to the behavior it verifies. The catalog is ordered from timer creation and list invariants through real pipe exchanges, malformed protocol input, exact wait-

handle capacity, capacity recovery, disconnect cleanup, concurrent API use, and the sample's stale-response rule.

Table 9. Automated test scenario catalog

Layer	Scenario	Verified behavior
Core	CreateTimerCreatesInactiveTimer	Confirms creation, a nonzero ID, and placement in the inactive list.
Core	CreateTimerAllocatesRunningIds	Confirms that successive creations receive monotonically increasing IDs.
Core	TimerIdWrapSkipsInvalidId	Confirms that counter wrap skips the reserved zero ID.
Core	TimerIdWrapSkipsIdAlreadyInUse	Confirms that a wrapped counter skips an ID that is still allocated.
Core	StartTimerMovesInactiveToActive	Confirms activation and the initial wait duration.
Core	StartTimerRejectsZeroDuration	Confirms that zero duration is rejected without mutating list state.
Core	InvalidTimerIdRejectedByCommands	Confirms that start, stop, and delete reject the reserved zero ID.
Core	WrongOwnerRejectedByCommands	Confirms that one

Layer	Scenario	Verified behavior
		client cannot manipulate another client's timer.
Core	SortedDeltaListUsesScheduledExpiryDeltas	Confirms out-of-order insertion and the resulting scheduled-expiry deltas.
Core	EqualExpiryTimersExpireTogether	Confirms zero-delta peers and one callback for each equal deadline.
Core	ProcessExpiredDoesNotExpireEarly	Confirms that a timer does not expire one millisecond early.
Core	ProcessExpiredMovesTimerBackInactive	Confirms the inactive transition and callback arguments after expiry.
Core	ProcessExpiredWithNullCallbackMovesTimerInactive	Confirms expiry processing still moves the timer inactive when no callback is registered.
Core	ProcessExpiredMultipleStaggeredTimersInOrder	Confirms that several due timers are reported once and in scheduled order.
Core	PartialExpiryLeavesLaterTimersActive	Confirms that removing the due head preserves the later timer and its

Layer	Scenario	Verified behavior
		delay.
Core	ExpiredTimerCanBeRestarted	Confirms that an expired timer can be started again.
Core	RestartActiveTimerWithShorterDuration	Confirms that a shorter restart moves the timer toward the active head.
Core	RestartActiveTimerWithLongerDuration	Confirms that a longer restart moves the timer behind earlier deadlines.
Core	StopActiveTimerMovesItInactive	Confirms that stop cancels an active schedule and returns the timer inactive.
Core	StopMiddleActiveTimerPreservesFollowingDelta	Confirms that stopping a middle active timer preserves the following timer's delta.
Core	StopInactiveTimerIsSuccessful	Confirms that stopping an inactive timer remains a successful idempotent operation.
Core	DeleteInactiveTimerRemovesIt	Confirms that deleting an inactive timer frees it and rejects repeated

Layer	Scenario	Verified behavior
		deletion.
Core	DeleteActiveTimerRemovesItAndKeepsOthers	Confirms active-head deletion without disturbing neighboring timers.
Core	DeleteMiddleActiveTimerPreservesFollowingDelta	Confirms middle active deletion and verifies the timer after it keeps the correct delta.
Core	GetNextWaitDurationWhenNoActiveTimer	Confirms the empty active-list scheduling result.
Core	RemoveTimersForPipeRemovesOnlyThatOwner	Confirms that disconnect cleanup removes only timers owned by the specified pipe.
Core	RemoveTimersForPipePreservesRemainingActiveDeltas	Confirms owner cleanup from the middle of the active list preserves remaining deltas.
Core	ExpiryArithmeticSaturatesAtUint64Max	Confirms that expiry arithmetic saturates instead of wrapping into the past.
Core	NearUint64MaxTimersRemainOrdered	Confirms ordering and delta calculations near UINT64_MAX.
Core	DeinitRemovesAllTimers	Confirms that deinitialization

Layer	Scenario	Verified behavior
		releases every timer and resets both lists.
Integration	CreateDeleteRoundTrip	Confirms the basic request, ACK, and delete path through the real pipe.
Integration	StartExpireDeleteRoundTrip	Confirms end-to-end start, expiry delivery, deletion, and an empty queue afterward.
Integration	StopActivePreventsExpiry	Confirms that stopping an active timer prevents a later expiry event.
Integration	StopInactiveRoundTrip	Confirms idempotent stop behavior through the public API and server.
Integration	RestartActiveUsesNewDuration	Confirms that restart replaces the old schedule and does not produce a stale second expiry.
Integration	DeleteActivePreventsExpiry	Confirms that deleting an active timer prevents a later expiry event.
Integration	MultipleTimersExpireThroughQueue	Confirms that one shared event can represent several queued timer IDs in

Layer	Scenario	Verified behavior
		FIFO order.
Integration	QueuedExpiriesKeepNotificationEventSignaled	Confirms that the notification event remains signaled until all queued expiries are drained.
Integration	NegativeAckStatusesFromUserPerspective	Confirms invalid ID, invalid duration, repeated delete, and empty-queue status through the public API.
Robustness	MalformedMagicRejected	Confirms magic rejection and successful use of the same connection afterward.
Robustness	MalformedVersionRejected	Confirms version rejection and connection recovery.
Robustness	MalformedTypeRejected	Confirms unsupported message-type rejection and connection recovery.
Robustness	MalformedTotalSizeRejected	Confirms validation of the exact received byte count.
Robustness	MalformedPayloadSizeRejected	Confirms validation of payload_size against the selected structure.

Layer	Scenario	Verified behavior
Robustness	MalformedTruncatedHeaderRejected	Confirms rejection of a message smaller than the protocol header.
Robustness	MalformedOversizedMessageRejected	Confirms rejection of extra bytes beyond the selected fixed-size command.
Robustness	MalformedPayloadTooSmallRejected	Confirms rejection when payload_size is smaller than the selected command requires.
Robustness	MalformedPayloadTooLargeRejected	Confirms rejection when payload_size is larger than the selected command allows.
Robustness	MalformedAckSentToServerRejected	Confirms that server-to-client ACK messages are rejected on the client-to-server path.
Robustness	MalformedExpirySentToServerRejected	Confirms that server-to-client expiry events are rejected on the client-to-server path.
Robustness	MaximumClientAndWaitHandleCapacity	Confirms 61 clients, one active timer, 64 wait handles, and rejection of an excess

Layer	Scenario	Verified behavior
		client.
Robustness	MaximumClientCapacityRecoversAfterDisconnect	Confirms that a released client slot restores the listener and admits a replacement client.
Robustness	DisconnectCleanupRemovesClientTimers	Confirms that closing a client pipe removes that client's active and inactive timers.
Robustness	NegativeAckOwnershipAndRawDuration	Confirms raw invalid duration, wrong-owner access, and unknown timer ID negative ACKs.
Robustness	ConcurrentPublicApiCalls	Confirms eight simultaneous first creates, unique IDs, and complete timer lifecycles.
Sample	SimultaneousResponseExpiries	Confirms one T3 or T4 transition and one stale response when both expiries are queued.

13. Current Limitations and Future Work

The current suite covers every documented user-visible scenario, but it does not inject failure into malloc, event creation, thread creation, or selected Windows API calls. Deterministic tests

for disconnects in the middle of an ACK or expiry write would further strengthen cleanup behavior, and retry after an initially unavailable server deserves a dedicated lifecycle test.

Version 1 also leaves service hosting, security descriptors, telemetry, benchmarking, and soak testing outside its scope. The server intentionally waits for writes to complete because traceable control flow matters more than throughput in this example. A production design could adopt fully overlapped writes or IOCP after measurements showed that the simpler event-wait model had become the limiting factor.

POSITIONING: This project is a readable RTOS-style example rather than a commercial timer service. Its value comes from making ownership, scheduling, protocol validation, and wait-handle behavior explicit enough to inspect in code and observe in live traces. Production throughput features are intentionally deferred until measurements justify their complexity.

14. AI-Assisted Development Workflow

This project was intentionally developed using OpenAI Codex as part of the engineering process. The objective was not simply to accelerate implementation, but to evaluate how AI-assisted development can support the design, implementation, review, testing, and documentation of a non-trivial systems software project.

AI-generated output was treated as a proposal rather than as authoritative implementation. Every accepted suggestion was reviewed for correctness, consistency with the documented architecture, coding style, synchronization rules, ownership model, and API contract before incorporation into the project.

Throughout development, architectural decisions, API design, concurrency model, protocol definition, verification strategy, and final technical decisions remained under human control. Codex was used to explore alternative designs, review implementation ideas, suggest test scenarios, improve documentation, and identify opportunities for simplification. Every accepted contribution was reviewed, validated, and, where necessary, revised before becoming part of the project.

Table 10. AI-Assisted development responsibilities

Engineering Activity	Codex Contribution	Engineer Responsibility
System architecture	Discussed architectural alternatives and design tradeoffs	Defined the overall architecture, component responsibilities, and interfaces
Client API	Suggested naming and consistency improvements	Designed the API contract, request/ACK handling, receiver thread, and synchronization
Server implementation	Assisted with implementation ideas, refactoring, and review	Designed the server architecture, scheduling, synchronization, and request processing
Timer core	Suggested implementation of refinements and edge cases	Designed scheduling algorithms, timer lifecycle, and ownership rules
Documentation	Assisted with wording, organization, and editing	Produced the technical content and ensured accuracy
Testing	Suggested additional test cases and corner conditions	Designed the verification strategy and validated correctness
Code review	Identified potential defects, inconsistencies, and simplification opportunities	Evaluated recommendations and made all final engineering decisions

Appendix A. Project Layout

The repository separates public contracts, timer algorithms, communication endpoints, executable behavior, and verification. Table 11 identifies the file that owns each responsibility so a reader can move from a design concept to its implementation without searching the entire solution.

Table 11. Project file layout

Path	Role in the implementation
include/vdr_timer_manager_protocol.h	Defines shared constants, numeric enum values, protocol structures, and directional message unions.
include/vdr_timer_manager_api.h	Declares the public timer API and documents event-handle ownership.
src/vdr_timer_core.h and src/vdr_timer_core.c	Defines Timer_type and implements active and inactive list algorithms.
src/client/vdr_timer_manager_api.c	Implements the pipe client, receiver thread, ACK handoff, state accessor, and expiry queue.
src/server/vdrapkin_timer_manager_server.c	Implements the console server, wait loop, state accessor, command dispatch, and expiry sending.
samples/sample_client.c	Implements the connection state machine, colored traces, shutdown accessor, and self-test mode.
tests/timer_core_tests.c	Implements 24 deterministic timer-core scenarios and accessor-controlled callback records.
tests/client_api_integration_tests.c	Implements seven API and server integration scenarios.
tests/robustness_tests.c	Implements malformed-protocol, capacity, and concurrency scenarios.
docs/timer_manager_design.md	Records the maintained design decisions and behavior contract.
CMakeLists.txt	Defines build targets, warning policy, and CTest registration.

Appendix B. Key Constants

The constants in Table 12 bind the protocol identity and Windows wait-set geometry. Changing a protocol constant requires both endpoints to be rebuilt, while changing a capacity constant also requires the wait-handle arithmetic and robustness expectations to be reviewed together.

Table 12. Key constants

Constant	Value	Purpose
PIPE_NAME	\\.\pipe\vdrapkinTimerManager	Names the endpoint that both client and server open.
TIMER_PIPE_MAGIC	0x544D4752	Identifies a message as Timer Manager protocol data.
TIMER_PROTOCOL_VERSION	1	Rejects an endpoint built for a different protocol revision.
TIMER_ID_INVALID	0	Reserves a value that can report failure when no timer ID exists.
TIMER_SERVER_MAX_WAIT_HANDLES	64	Matches the Windows WaitForMultipleObjects handle limit.
TIMER_SERVER_MAX_CLIENTS	61	Preserves slots for the listener, waitable timer, and shutdown event.

Appendix C. Design Invariants

A timer exists in exactly one list at a time. Active timers remain ordered by scheduled expiry, expired or stopped timers return to inactive state, and the core invokes the expiry callback only after that transition is complete. Timer ID zero is reserved permanently and is skipped when the running counter wraps.

Ownership is enforced throughout command processing. A command cannot manipulate another client's timer, each valid command produces one generic ACK, and disconnect cleanup deletes every timer owned by that pipe. Each client pipe has at most one outstanding asynchronous read, while each client API process permits at most one command and matching ACK exchange at a time.

The notification-event state represents whether the expiry queue is empty. The event remains signaled while at least one timer ID is queued and resets only after the application removes the final queued ID. On the server, the wait set never exceeds 64 handles because admission stops at 61 connected clients and reserves one handle for the listener, one for the waitable timer, and one for shutdown.

Appendix D. Representative Execution Traces

The following excerpts were captured from fresh runs of the current C11 Debug executables. Each block is an exact, ordered subset of the captured output. The source-file path and line-number prefix added by PRINTOUT_TRACE is omitted for readability, and lines outside the behavior described by the subsection are not reproduced. Every retained trace message, field, value, and relative ordering is unchanged. Handle values, timer identifiers, timestamps, and measured wait durations belong to this capture and may differ in later runs.

D.1 Successful Timer Lifecycle

The first excerpt follows one timer through creation, start, asynchronous expiry delivery, and deletion. The application calls the public API for each command. The client API sends the corresponding request, waits for the matching ACK, and later queues the independent expiry indication for the application.

```
[integration] StartExpireDeleteRoundTrip: begin
[client-api] connected to pipe \\.\pipe\vdrapkinTimerManager
handle=00000000000000B4
[client-api] pipe read mode set to message mode
[client-api] initialization complete
receiver_thread=00000000000000C0
[client-api] sending request: request=1 type=CREATE_TIMER_REQ
payload=0
[client-api] request written: request=1 bytes=16; waiting for ACK
[client-api] receiver read message type=ACK_RSP request=1 bytes=32
[client-api] received ACK: request=1 status=OK(0) timer=1
[client-api] completed request: request=1 type=CREATE_TIMER_REQ
status=OK(0) timer=1
[client-api] sending request: request=2 type=START_TIMER_REQ
payload=16
[client-api] request written: request=2 bytes=32; waiting for ACK
[client-api] receiver read message type=ACK_RSP request=2 bytes=32
[client-api] received ACK: request=2 status=OK(0) timer=1
[client-api] completed request: request=2 type=START_TIMER_REQ
status=OK(0) timer=1
[client-api] receiver read message type=TIMER_EXPIRED_EVT request=0
bytes=24
[client-api] queued timer expiry indication: timer=1
[client-api] application retrieved expired timer: timer=1
[client-api] sending request: request=3 type=DELETE_TIMER_REQ
payload=8
[client-api] request written: request=3 bytes=24; waiting for ACK
```

```
[client-api] receiver read message type=ACK_RSP request=3 bytes=32
[client-api] received ACK: request=3 status=OK(0) timer=1
[client-api] completed request: request=3 type=DELETE_TIMER_REQ
status=OK(0) timer=1
[integration] StartExpireDeleteRoundTrip: pass
```

The server-side excerpt for the same timer shows validation before payload processing, command dispatch to the timer core, ACK transmission, waitable-timer programming, and expiry indication delivery.

```
[server][command] begin pipe=00000000000000BC bytes=16
raw_type=CREATE_TIMER_REQ request=1
[server][validate] begin bytes_read=16
[server][validate] pass: request=1 type=CREATE_TIMER_REQ payload=0
[server][command] dispatch CreateTimer request=1
[server][command] core CreateTimer done status=OK(0) timer=1
[server][ack] sent ACK request=1 status=OK(0) timer=1
[server][command] rearming waitable timer after request=1
[server][timer-arm] no active timers; waitable timer canceled
[server][command] complete request=1 status=OK(0) timer=1
[server][command] begin pipe=00000000000000BC bytes=32
raw_type=START_TIMER_REQ request=2
[server][validate] begin bytes_read=32
[server][validate] pass: request=2 type=START_TIMER_REQ payload=16
[server][command] dispatch StartTimer request=2 timer=1
duration=100
[server][command] core StartTimer done timer=1 duration=100
status=OK(0)
[server][ack] sent ACK request=2 status=OK(0) timer=1
[server][command] rearming waitable timer after request=2
[server][timer-arm] waitable timer armed duration=97 ms
now=942657861
[server][command] complete request=2 status=OK(0) timer=1
[server][timer-expiry] begin
[server][timer-expiry] sending indication timer=1
pipe=00000000000000BC
[server][expiry-send] sent timer expiry indication timer=1
[server][timer-expiry] indication complete timer=1
[server][timer-expiry] rearming waitable timer after expiry
processing
[server][timer-arm] no active timers; waitable timer canceled
[server][timer-expiry] complete
[server][command] begin pipe=00000000000000BC bytes=24
raw_type=DELETE_TIMER_REQ request=3
[server][validate] begin bytes_read=24
[server][validate] pass: request=3 type=DELETE_TIMER_REQ payload=8
```

```
[server][command] dispatch DeleteTimer request=3 timer=1
[server][command] core DeleteTimer done timer=1 status=OK(0)
[server][ack] sent ACK request=3 status=OK(0) timer=1
[server][command] rearming waitable timer after request=3
[server][timer-arm] no active timers; waitable timer canceled
[server][command] complete request=3 status=OK(0) timer=1
```

The requested duration is 100 milliseconds, while the waitable timer is armed for 97 milliseconds in this capture. The three-millisecond difference represents time spent processing the request before the server recalculates the remaining delay from the timer's scheduled expiry.

D.2 Simultaneous Response Expiries

The sample client's deterministic self-test starts T3 and T4 with the same duration. Both expiry indications can therefore be queued before the application handles either one. The first retrieved expiry causes the applicable state transition. The second expiry is stale in the new state and is ignored. This scenario verifies that one notification event can represent multiple queued timer identifiers without causing two state transitions.

```
[sample-client][self-test] simultaneous response expiry: begin
[sample-client][timer] creating T1
[sample-client][timer] created T1 timer=1
[sample-client][timer] creating T2
[sample-client][timer] created T2 timer=2
[sample-client][timer] creating T3
[sample-client][timer] created T3 timer=3
[sample-client][timer] creating T4
[sample-client][timer] created T4 timer=4
[sample-client][timer] creating T5
[sample-client][timer] created T5 timer=5
[sample-client][timer] starting T3 timer=3 duration_ms=100
[sample-client][timer] started T3 timer=3
[sample-client][timer] starting T4 timer=4 duration_ms=100
[sample-client][timer] started T4 timer=4
[client-api] queued timer expiry indication: timer=3
[client-api] queued timer expiry indication: timer=4
[client-api] application retrieved expired timer: timer=3
[sample-client][timer-expiry] T3 expired timer=3
[sample-client][message][RX] CONNECTION_ACCEPT
[sample-client][timer] stopping T4 timer=4
[sample-client][timer] stopped T4 timer=4
[sample-client][timer] stopping T2 timer=2
[sample-client][timer] stopped T2 timer=2
```

```
[sample-client][state] WAITING_FOR_CONNECTION_ESTABLISHMENT ->
CONNECTED
[sample-client][timer] starting T5 timer=5 duration_ms=13000
[sample-client][timer] started T5 timer=5
[client-api] application retrieved expired timer: timer=4
[sample-client][timer-expiry] T4 expired timer=4
[sample-client][timer] ignoring stale T4 expiry in state=CONNECTED
[sample-client][self-test] simultaneous response expiry: pass
```

Stopping T4 after its expiry remains valid because expiry has already returned the timer to the inactive list. The queued T4 indication cannot be withdrawn, so the state machine checks its current state before assigning meaning to the indication.

D.3 Malformed Request Rejection and Recovery

The final excerpt shows a request with an invalid protocol magic value. The server returns PROTOCOL_ERROR with TIMER_ID_INVALID and performs no timer operation for that request. A subsequent valid request on the same connection succeeds, which demonstrates that rejecting one malformed message does not corrupt command processing or require the client to reconnect.

```
[server][command] begin pipe=00000000000000BC bytes=16
raw_type=CREATE_TIMER_REQ request=100
[server][validate] begin bytes_read=16
[server][validate] failed: magic=0x544D4753 version=1
[server][ack] sent ACK request=100 status=PROTOCOL_ERROR(5) timer=0
[server][command] rejected malformed request=100
[server][command] begin pipe=00000000000000BC bytes=16
raw_type=CREATE_TIMER_REQ request=110
[server][validate] begin bytes_read=16
[server][validate] pass: request=110 type=CREATE_TIMER_REQ
payload=0
[server][command] dispatch CreateTimer request=110
[server][command] core CreateTimer done status=OK(0) timer=1
[server][ack] sent ACK request=110 status=OK(0) timer=1
[server][command] rearming waitable timer after request=110
[server][command] complete request=110 status=OK(0) timer=1
[server][command] begin pipe=00000000000000BC bytes=24
raw_type=DELETE_TIMER_REQ request=111
[server][validate] begin bytes_read=24
[server][validate] pass: request=111 type=DELETE_TIMER_REQ
payload=8
[server][command] dispatch DeleteTimer request=111 timer=1
[server][command] core DeleteTimer done timer=1 status=OK(0)
[server][ack] sent ACK request=111 status=OK(0) timer=1
```

```
[server][command] rearming waitable timer after request=111
[server][command] complete request=111 status=OK(0) timer=1
```